Lecture 37 Practice Problems: Neural Network Training Foundations

This problem set consists of 15 rigorous analytical, mathematical, and implementation problems designed to test deep conceptual understanding of backpropagation, loss functions, gradient descent dynamics, activation behaviors, and modern adaptive optimizers.

Problem 1: Forward Pass Output Computation

A single artificial neuron receives feature inputs $x = [2.0, -1.0, 3.0]$ with synaptic weights $w = [0.4, -0.6, 0.2]$ and bias $b = -0.5$. The neuron employs the Logistic Sigmoid activation function. Compute the pre-activation linear scalar z and the final activation prediction $\hat{y}$.

Problem 2: Outlier Sensitivity in Regression Losses

A regression dataset has true target $y = 10.0$. Due to sensor malfunction, an extreme outlier produces a predicted output $\hat{y} = 60.0$.

1. Compute the loss penalty using Mean Squared Error (MSE with $\frac{1}{2}$ scaling).
2. Compute the loss penalty using Mean Absolute Error (MAE).
3. Compute the Huber Loss with threshold $\delta = 2.0$. Explain which loss function prevented gradient explosion and why.

Problem 3: Binary Cross-Entropy Penalty Analysis

For a binary classification task, sample A has true ground truth $y_A = 1$ and model prediction $\hat{y}_A = 0.95$. Sample B has ground truth $y_B = 1$ but model prediction $\hat{y}_B = 0.05$.

1. Compute the Binary Cross-Entropy loss for Sample A and Sample B.
2. What is the ratio of penalties between the two predictions? Explain the optimization consequence.

Problem 4: Softmax Probability Distribution & Categorical Cross-Entropy

A 3-class neural network generates raw unnormalized logits $z = [2.0, 1.0, 0.1]$ for an image of a 'Cat'. The ground truth one-hot vector is $y = [1, 0, 0]$.

1. Calculate the normalized Softmax probability vector $\hat{y}$.
2. Compute the Categorical Cross-Entropy loss $\mathcal{L}$.

Problem 5: Analytic Derivation of Softmax Derivative

Prove analytically that the derivative of the Softmax activation function $\hat{y}_i = \frac{e^{z_i}}{\sum_k e^{z_k}}$ with respect to logit z_j is:

$$\frac{\partial \hat{y}_i}{\partial z_j} = \hat{y}_i(\delta_{ij} - \hat{y}_j)$$

where δ_{ij} is the Kronecker delta ($\delta_{ij} = 1$ if $i = j$, else 0).

Problem 6: Single-Step Gradient Descent Update

A linear neuron has current weight $w = 2.5$ and bias $b = 0.8$. On a single training sample $(x = 1.5, y = 4.0)$, the neuron predicts $\hat{y} = wx + b$. Using squared error loss $\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$ and learning rate $\eta = 0.1$, compute the updated weight w_{new} and bias b_{new}.

Problem 7: Backpropagation via Chain Rule

Consider a composite scalar function:

$$z = wx + b, \quad a = \text{ReLU}(z), \quad \mathcal{L} = \frac{1}{2}(y - a)^2$$

Given inputs $x = 3.0, y = 5.0, w = 1.0, b = -1.0$:

1. Compute forward quantities $z, a, \mathcal{L}$.
2. Evaluate partial derivatives $\frac{\partial \mathcal{L}}{\partial a}$, $\frac{\partial a}{\partial z}$, $\frac{\partial z}{\partial w}$.
3. Assemble the composite gradient $\frac{\partial \mathcal{L}}{\partial w}$ via Chain Rule.

Problem 8: Mathematical Proof of Sigmoid Gradient Upper Bound

Prove mathematically using differential calculus that the derivative of the Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ achieves its absolute global maximum strictly at $z = 0$, with maximum value $\sigma'(0) = 0.25$. Hence prove that across an 8-layer network, the gradient shrinks by a factor of at least $(0.25)^8$.

Problem 9: The Dying ReLU Pathology

A neuron in hidden layer 2 has parameters $w = [0.5, -0.2]$ and bias $b = -2.0$. For all input feature vectors x in the training dataset, $x_1 \in [0, 1]$ and $x_2 \in [0, 1]$.

1. Prove that pre-activation $z < 0$ for all valid dataset inputs.

2. Explain why this neuron will never update its weights under standard backpropagation.

Problem 10: Leaky ReLU Gradient Flow

Repeat Problem 9 using Leaky ReLU with $\alpha = 0.01$. Show how the non-zero subgradient prevents permanent neuron deactivation.

Problem 11: Training Epoch, Batch, and Step Calculations

An industrial autonomous vehicle dataset contains $N = 128,000$ camera frames. The model is trained on an 8-GPU cluster using Mini-Batch Gradient Descent with an effective batch size of $B = 256$ for $E = 50$ epochs.

1. How many iterations (weight updates) are executed per epoch?
2. How many total weight updates are performed during full training?
3. If each iteration takes $120\ \text{ms}$ of compute time, what is the total training duration in hours?

Problem 12: Gradient Variance Comparison (BGD vs SGD vs MBGD)

Explain mathematically why the variance of the gradient estimator in Mini-Batch Gradient Descent scales as $\frac{\sigma^2}{B}$, where σ^2 is the variance of a single sample gradient and B is the batch size. Discuss the implications for training stability.

Problem 13: Momentum Numerical Simulation

A parameter has initial velocity $v_0 = 0$ and initial weight $\theta_0 = 5.0$. The momentum hyperparameter is $\beta = 0.9$ and learning rate is $\eta = 0.05$. At time steps $t = 1, 2, 3$, the computed gradients are consistently $g_1 = 2.0, g_2 = 2.0, g_3 = 2.0$. Compute the velocity v_t and updated parameter θ_t for all three steps.

Problem 14: RMSProp Adaptive Step Scaling

Two parameters θ_A and θ_B experience persistent gradients $g_A = 10.0$ (steep ravine) and $g_B = 0.1$ (flat plateau). Using RMSProp with $\beta = 0.9, \epsilon = 10^{-8}, \eta = 0.01$, and initial $s_0 = 0$:

1. Compute the second moment s_1 for both parameters.
2. Compute the effective step size $\frac{\eta}{\sqrt{s_1 + \epsilon}} g_1$.
3. Demonstrate how RMSProp automatically normalizes the updates across both parameters.

Problem 15: Full Adam Optimizer Step Calculation

Given initial parameter $\theta_0 = 1.0$, learning rate $\eta = 0.001$, hyperparameters $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$. At time step $t = 1$, the evaluated gradient is $g_1 = 0.5$.

1. Compute the biased first moment m_1 and second moment v_1.
2. Compute the bias-corrected moments $\hat{m}_1$ and $\hat{v}_1$.
3. Compute the final updated parameter θ_1.\n